Josh Collinsworth

MOVE

Let's learn SvelteKit by building a static Markdown blog from scratch

PUBLISHED: December 27, 2021
UPDATED: May 20, 2022

If you've ever recommended a favorite book, movie, or TV show to a friend who didn't enjoy it at all, you know exactly how I felt when I saw this message from a colleague:

> *man I just tried to hello world SvelteKit and it is NOT for me*

I love Svelte, and I've talked plenty about how much I've enjoyed rewriting this site using SvelteKit. So my first reaction, of course, was disappointment. What went wrong?

But as I considered their reaction, I realized: it was valid.

Amazing though it is, SvelteKit isn't the simplest thing you could use as a static site generator.

Other tools like Astro, Eleventy, or Gatsby will likely get you up and running faster, since that's their sole focus. (Even the SvelteKit docs make this point.)

However, I still feel SvelteKit is worthy of serious consideration. Its capabilities are much more broad and flexible than SSGs, and there's plenty unique to love about its both developer and user experiences. So this post will be aimed at smoothing out that initial learning curve, to help you understand and take advantage of SvelteKit's wide range of possibilities.

What we'll cover, and what to know first

This is a walkthrough of using SvelteKit to set up a prerendered static blog with Markdown, Sass and an API. (That's how this site is built, by the way.)

This is *also* an intro to SvelteKit itself, with a good overview of SvelteKit fundamentals that should be transferable to *any* project.

Finally, this is *fun*!*

**My opinion; citation needed*

MOVE

You'll get the most out of this if you:

- Know the fundamentals of front-end development and JavaScript;
- Have at least a basic understanding of Svelte already. (You can probably follow along regardless, but I'd recommend the Svelte tutorial first if you're brand-new);
- Know the basics of both Markdown and Sass;
- Understand the basics of fetching JSON from an API; and
- Know how to install packages with npm, and have npm installed already.

Finally, if you just want to skip ahead and see the finished product, you can check out this repo or copy it—though I have a proper, more fleshed-out SvelteKit starter here.

Let's get started!

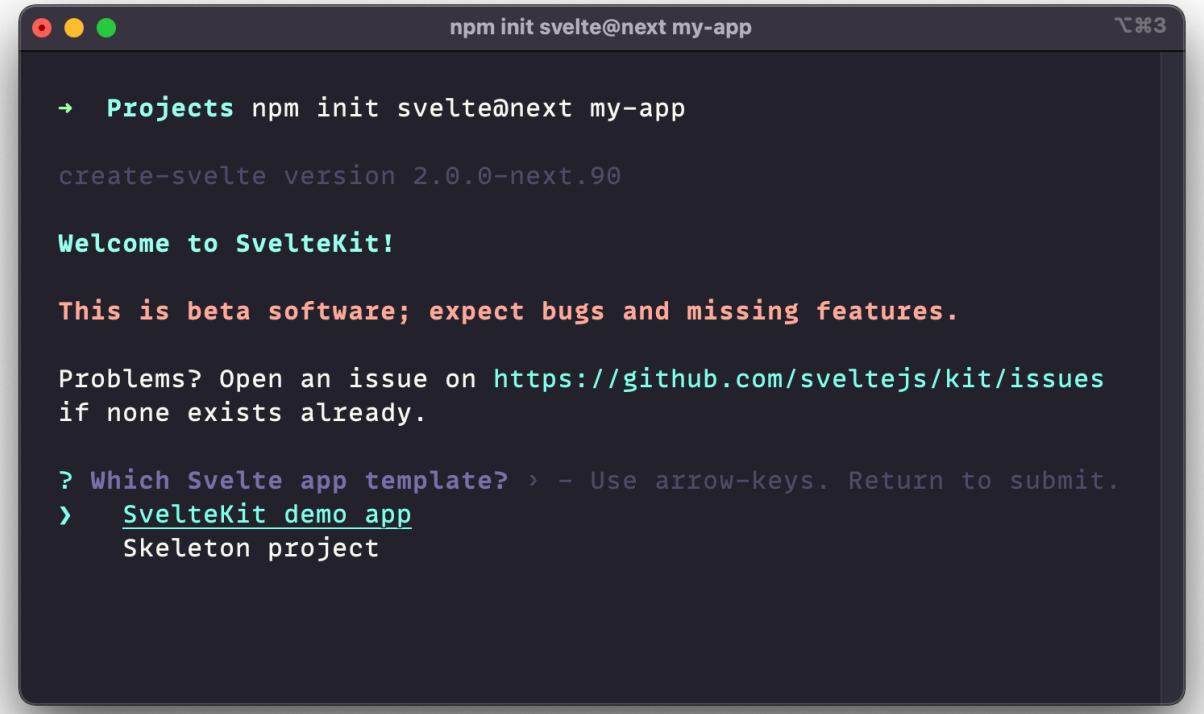
Creating a new SvelteKit app

To generate a new SvelteKit project, run this in your terminal (naturally, you can change `my-app` to whatever other title you prefer):

Terminal

```
npm init svelte@next my-app
```

When you run that `init` command, SvelteKit will ask you some questions about your project and how you want it set up:



We'll select the "Skeleton project" option.

SIDE NOTE:

If this is your very first time using SvelteKit, feel free to choose "SvelteKit demo app," poke around a bit to get a sense of how things work, then come back. It's a good intro, but it comes with several files and styles that would be a lot to undo as a starter.

The setup tool will also ask for your preferences on TypeScript, ESlint, and Prettier. For simplicity's sake, I won't use TypeScript, but you're welcome to if you like. (Svelte works quite well with it.)

Once you've made your choices, after a moment of installation, you should see `Your project is ready!` appear in your terminal, along with next steps and helpful links:

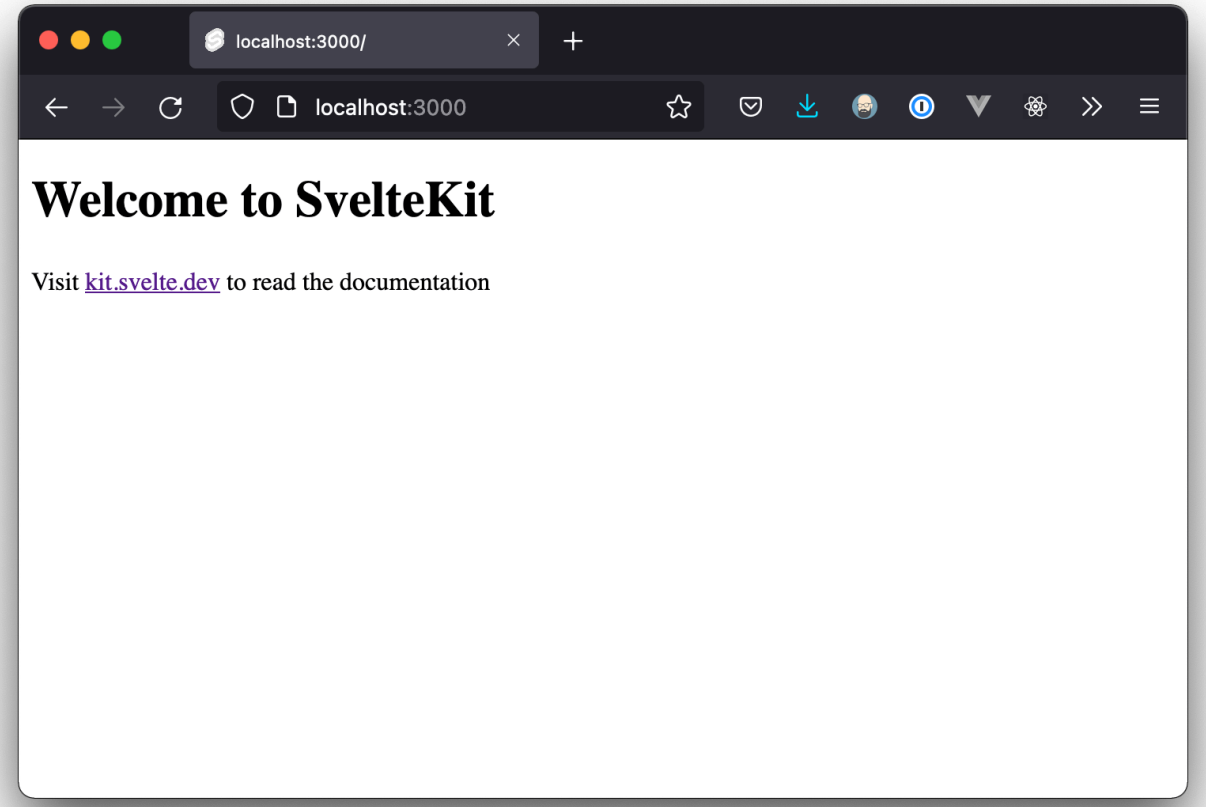
```
joshcollinsworth@Joshs-MacBook-Pro:~/Projects/my-app

Your project is ready!
✔ ESLint
  https://github.com/sveltejs/eslint-plugin-svelte3
✔ Prettier
  https://prettier.io/docs/en/options.html
  https://github.com/sveltejs/prettier-plugin-svelte#options

Install community-maintained integrations:
  https://github.com/svelte-add/svelte-adders

Next steps:
  1: cd my-app
  2: npm install (or pnpm install, etc)
  3: git init && git add -A && git commit -m "Initial commit" (optional)
  4: npm run dev -- --open

To close the dev server, hit Ctrl-C

Stuck? Visit us at https://svelte.dev/chat
```

At this point, let's open up the project in VS Code (or your text editor of choice). Then in the terminal, install the dependencies, and then run the dev server:

Terminal

```
npm install
npm run dev -- --open
```

Complete those commands, and our new site will be running in a browser window!



The skeleton project is definitely unexciting. There isn't even any CSS yet. But that's ok; it gives us the freedom to style our app however we like, without the need to remove or work around any existing code.

This is our blank canvas, and we'll get some paint on it soon enough.

Routing in SvelteKit

Like many frameworks (and for that matter, PHP), SvelteKit has a directory-

Like many frameworks (and for that matter, PHP), SvelteKit has a directory based approach to routing and pages.

Any component inside `src/routes` becomes a page at that same relative route.

- `index.svelte` is the homepage (`/`)
- A file named `about.svelte` would become the `/about` page
- `blog/index.svelte` would be be the `/blog` page
- `blog/some-post.svelte` becomes `/blog/some-post`

Adding pages

For starters, just to get some pages on our site, let's create `about.svelte` and `contact.svelte` files inside of `src/routes`. (We can always delete or modify them later.) We'll also create a `blog` folder, with an `index.svelte` file inside.

Within each of those files, let's toss a quick heading, and maybe some text, just so we can see a little content.

SIDE NOTE:

Svelte files can just contain plain HTML.

Svelte

```
<!— about.svelte —>
<h1>Hi, I'm Josh!</h1>

<p>This is my about page.</p>
```

Svelte

```
<!— contact.svelte —>
<h1>Get in touch</h1>

<p><a href="mailto:me@my.tld">Email me!</a></p>
```

Svelte

```
<!— blog/index.svelte —>
<h1>Blog</h1>

<p>My blog posts will go here eventually…</p>
```

Our `src/routes` folder should now look like this:

Project files

```
src
└ routes
  ├ blog
  │ └ index.svelte
  ├ about.svelte
  ├ contact.svelte
  └ index.svelte
```

At this point, you could visit `/contact`, `/about`, and/or `/blog` to see the pages we just created. There's not much there yet, of course, but it's nice to at least see that routing is taken care of for us automatically, just by placing the files in the `routes` folder.

Adding a layout

Most sites and apps have some elements that are reused on every page (a `<header>` and `<footer>` being the most common and obvious examples). Since adding those to every page would be tedious and error-prone, SvelteKit (like many other frameworks) conveniently lets us define layouts.

Inside of `src/routes`, make a new file named `__layout.svelte`

Project files

```
src
└ routes
  └ __layout.svelte
```

This name is a SvelteKit convention. SvelteKit automatically checks `routes` (and all its subdirectories) for a `__layout.svelte` file. That layout will "wrap" all the content loaded from that route and its sub-routes.

A layout becomes a parent component, and the current page will be slotted in as its child component.

So a layout at the top level of the `routes` folder will apply globally, on every page.

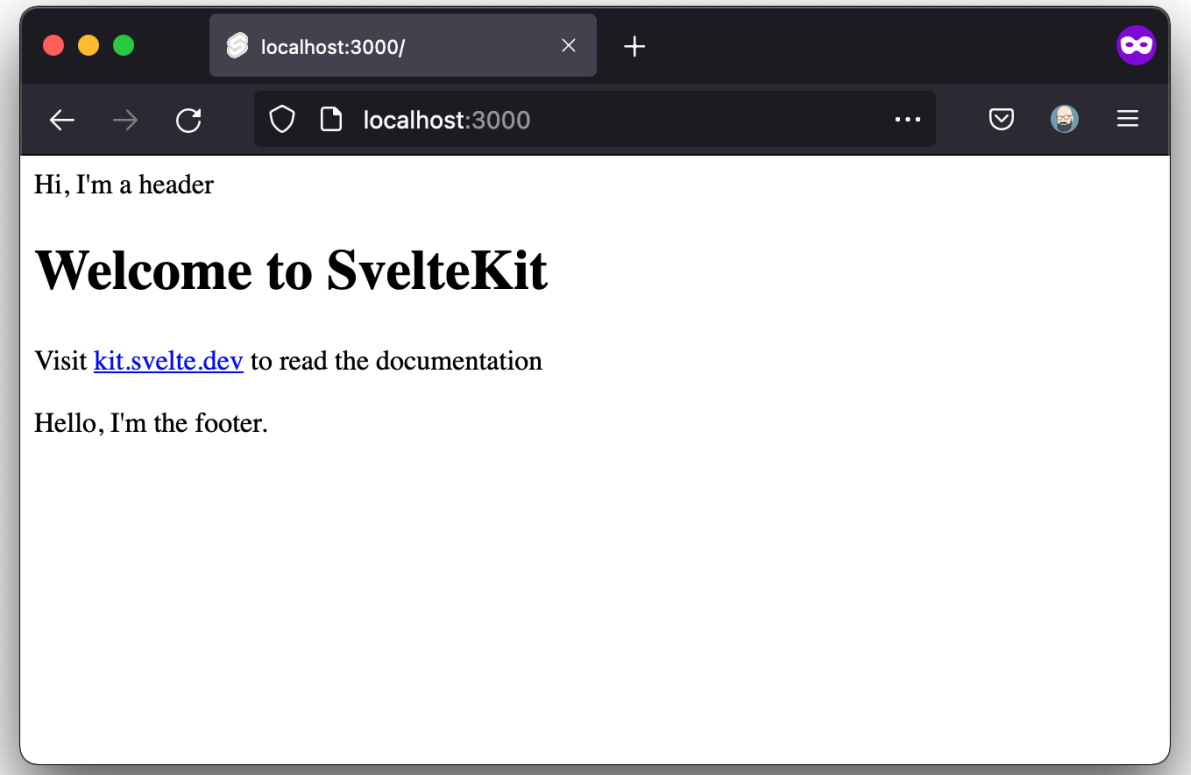
It's easier to show than tell, so open up our new `__layout.svelte` file, and let's add a placeholder header and footer:

Svelte

```svelte
<!— __layout.svelte —>
<header>Hi, I'm a header</header>

<main>
  <slot />
</main>

<footer>Hello, I'm the footer.</footer>
```

Save that, and you should now see that header text above the page's contents,

and the footer text below.



Creating components

Manually typing routes in the browser isn't great, so let's add some navigation. In the process, we'll create our first component.

First, create a new folder called `lib` inside of `src`. (`lib` is another SvelteKit convention; more on that in a moment.)

Inside `src/lib`, we'll create a folder named `components`, just for organizational

purposes. And inside of *that*, we'll create a new component named `Header.svelte`:

Project files

```
src
├ routes
└ lib
  └ components
    └ Header.svelte
```

SIDE NOTE:

You don't have to capitalize component names, or keep components in their own folder, but both are generally preferred conventions.

Inside our new `Header` component, let's place some proper (albeit basic) navigation:

Svelte

```svelte
<!— Header.svelte —>
<header>
  <a href="/">Home</a>

  <nav>
    <ul>
      <li>
        <a href="/blog">Blog</a>
      </li>
      <li>
        <a href="/about">About</a>
      </li>
      <li>
        <a href="/contact">Contact</a>
      </li>
    </ul>
  </nav>
```

```
</header>
```

Save that, and let's go back to `__layout.svelte` to use it!

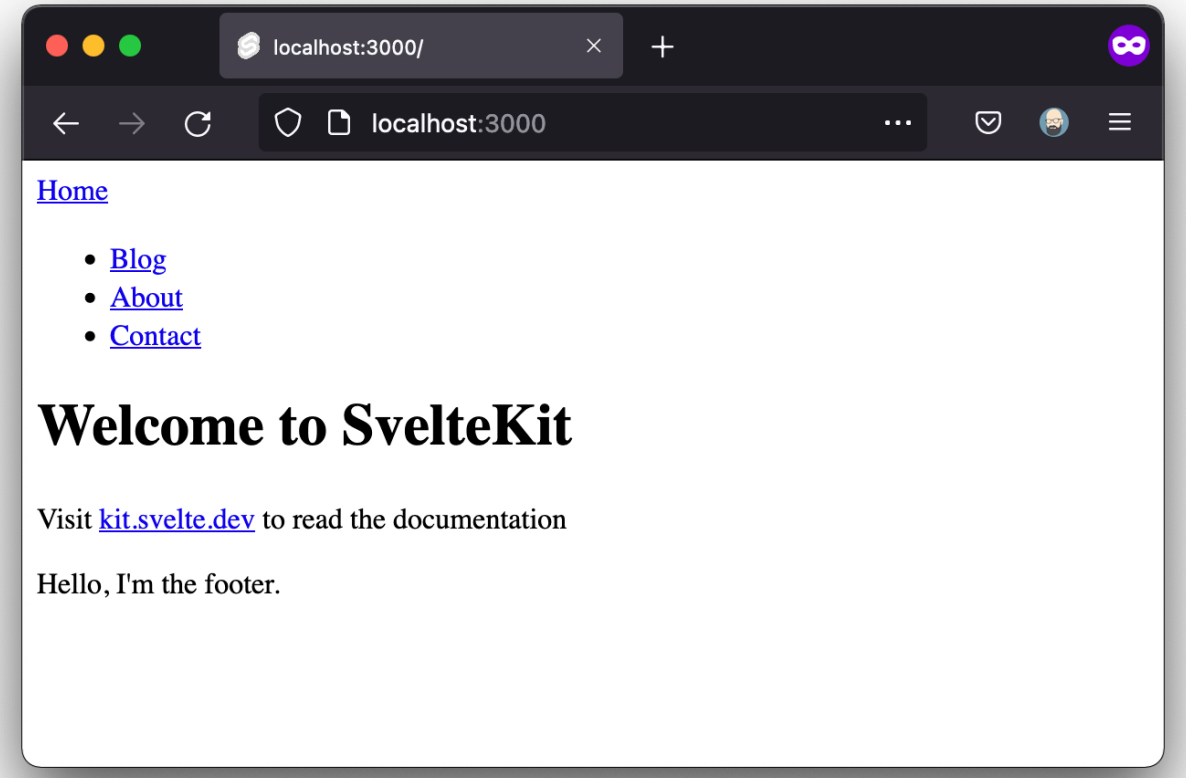
To use a component, we first need to `import` it, which means we'll need to add a `<script>` tag to our layout. (I personally prefer it at the top of the file, but Svelte gives you the freedom to arrange components however you like.) Once imported, we can replace our placeholder header with the component:

Svelte

```
<!— __layout.svelte —>
<script>
  import Header from '$lib/components/Header.svelte'
</script>

<Header />
<!— The rest of the HTML here —>
```

Notice the import path. `$lib` is a handy alias that goes directly to `src/lib`, saving you from typing tedious relative paths. (`$lib` is the only alias SvelteKit ships with, but you can add your own, from `jsconfig.json`.)

Anyway, now we should see something *slightly* more user-friendly in our browser:



The styling still isn't great, but it's usable. We can click between pages now, which will make testing things out a lot easier.

Component styling

Svelte component styles are scoped by default. That makes it easy to write CSS without fear of it bleeding out and having side effects anywhere outside the component.

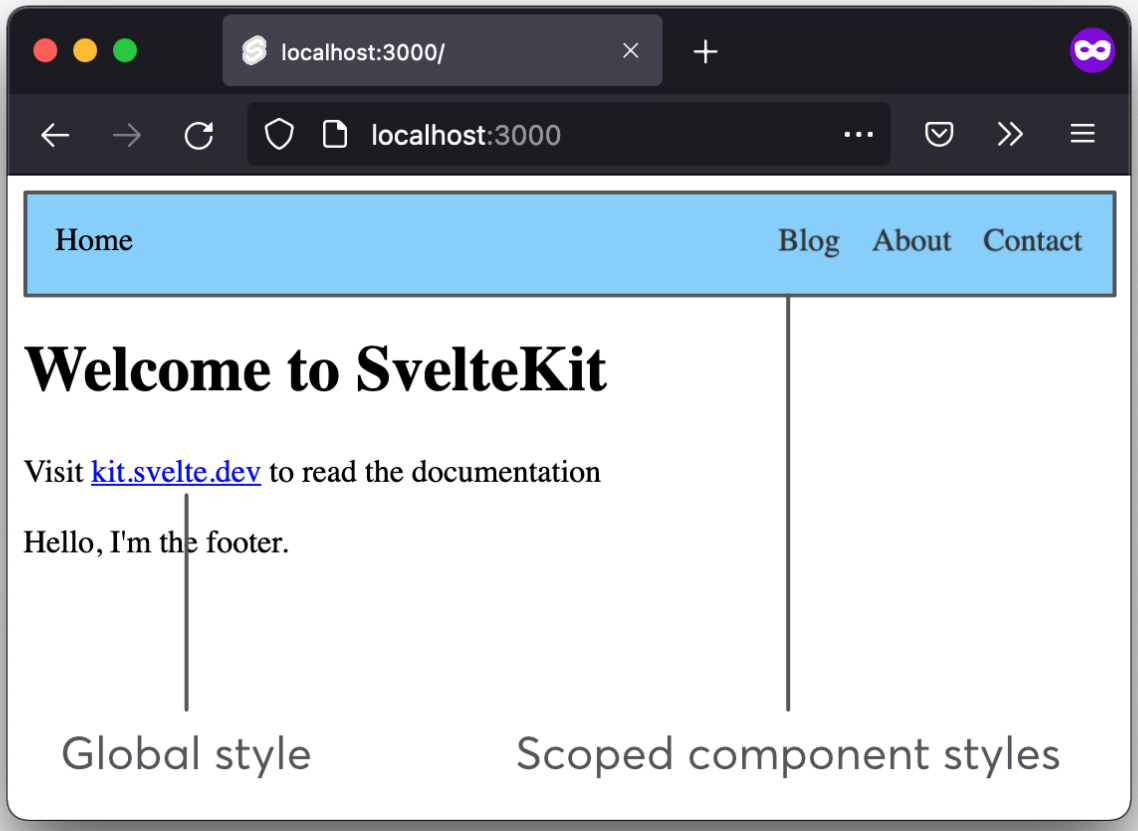
To try it out in [illegible] add a [illegible] tag. (It could go anywhere, but I

To try it out, in `Header.svelte`, add a `<style>` tag. (It could go anywhere, but I prefer to put it at the end.)

Svelte

```
<!— Header.svelte —>

<!— ... The rest of the file's contents here —>

<style>
  header {
    padding: 1rem;
    background: lightskyblue;
    display: flex;
    flex-wrap: wrap;
    justify-content: space-between;
  }

  ul {
    margin: 0;
    list-style-type: none;
    display: flex;
    gap: 1rem;
  }

  a {
    text-decoration: none;
    color: inherit;
  }
</style>
```

If you save and take a look at the site now, you should see your Header component looking a little more header-like. You might *also* notice that the styling doesn't affect anything in our main page content. That's the power of scoping!



If you want to, you could repeat this process with a `<Footer />` component now, too. I'll skip over it, since it's just the same thing over again, but it's a good thing to do at some point.

The difference between components and pages

It might be a little confusing that we're jumping in and out of several `.svelte` files now, each with a different role. So just to clarify before moving on:

- Any component inside of `src/routes` becomes its own page (unless its name starts with an underscore);
- Otherwise, we're putting them in `src/lib/components`. They can be used independently, and don't get routes of their own.

Stylesheets in SvelteKit

Svelte lets you import stylesheets directly into components—including pages and layouts. There are other ways to load stylesheets, but I find that's the best. It's easy, for one thing, but more importantly, it *also* preprocesses them automatically, which we'll want when we add Sass shortly.

Create a new folder inside of `src/lib` for your styles, and add a `style.css` file inside it.

Project files

```
src
├ routes
└ lib
  ├ components
  └ styles
    └ style.css
```

SIDE NOTE:

There are no rules about how you structure things inside of `src/lib`*. Feel free to organize your folders however you like.*

You can add whatever CSS you prefer, but if you need a suggestion, here's a little bit of boilerplate that should begin to get things looking *slightly* better:

CSS

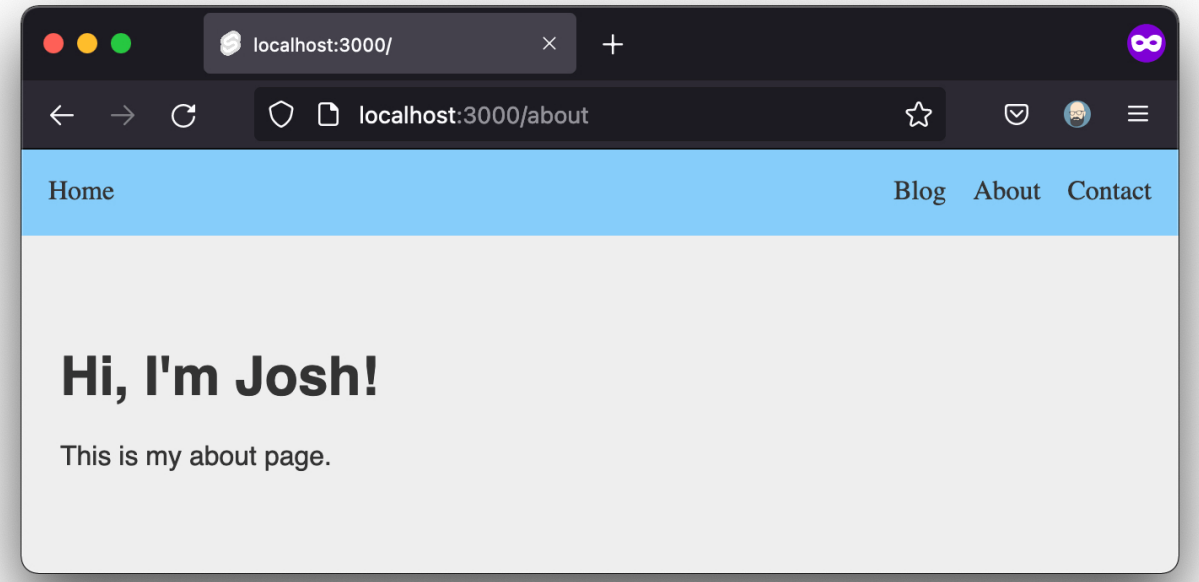
```
/* style.css */
```

```
body {
  margin: 0;
  background: #eee;
  color: #333;
  font-family: sans-serif;
}

main {
  padding: 1rem;
  margin: 2rem auto;
  max-width: 40rem;
}
```

Once we’ve got a stylesheet with some CSS in it, loading it globally is as easy as importing the file in our layout:

Svelte

```
<!— __layout.svelte —>
<script>
  import Header from '$lib/components/Header.svelte'
  import '$lib/styles/style.css'
</script>

<!— ...HTML here —>
```



MOVE

Since SvelteKit is server-rendered, if you view the page source, you'll even see our style has been added to the HTML, rather than being client-rendered. Neat!

SIDE NOTE:

Importing stylesheets in JavaScript files is not a web standard; it's just supported by some bundlers and build tools, popularized by React and Webpack.

Adding Sass to SvelteKit

One of the great things about Svelte is how comparatively easy it is to add external processors, thanks to `svelte-preprocess`.

> `svelte-preprocess` *automatically transforms the code in your Svelte templates to provide support for TypeScript, PostCSS, scss/sass, Less, and many other technologies.*

Our next step is to install both `svelte-preprocess` and `sass`, which we may as well do at the same time:

Terminal

```
npm i -D svelte-preprocess sass
```

SIDE NOTE:

This command installs the most modern version of Sass, which isn't compatible with all versions of Node. If you hit errors, you may need to upgrade your Node version, or install the older `node-sass` *instead.*

MOVE

MODIFYING THE SVELTE CONFIG

Next, open the `svelte.config.js` file, found in the project root.

As the name implies, `svelte.config.js` is the home for, well, configuration. It's where we'll make any needed changes to SvelteKit's options and compiler.

This change is minimal; we just need to import `svelte-preprocess`, and then add it to SvelteKit's list of preprocessors.

JavaScript

```javascript
// svelte.config.js
import sveltePreprocess from 'svelte-preprocess'

const config = {
  kit: { /* ...other kit options here already */ },

  preprocess: [
    sveltePreprocess(),
  ],
}
```

Note that you'll need to create the `preprocess` option; it's not there by default. (We'll add Markdown processing here later, too.)

Also note that you *can* pass an object with more granular options into the `sveltePreprocess()` function, but we won't for now. It knows what to do with Sass by default, so no further customization is needed.

SIDE NOTE:

It's worth mentioning the `scss.prependData` *option, which allows you to automatically inject code at the start of any Sass processing (such as Sass variables or an* `@import`*). You can read more in the* `svelte-preprocess` *docs; I just wanted to mention it since it's quite handy.*

One more quick, important thing to know: you'll want to stop and restart your dev server any time you make a config change.

Adding SCSS to component styles

Now that we've got both Sass and our preprocessor installed, you can use Sass inside any component's `<style>` block just by adding a `lang` attribute, like so:

Svelte

```
<style lang="scss">
  // We can write SCSS here!
</style>
```

Using global SCSS files

Linking to Sass files is actually exactly the same as linking to CSS files, like we've already covered. In fact, at this point, you could just rename the existing `style.css` file to `style.scss`. (You'll need to update the `import` in our layout, too, but that's it.)

Svelte

```
<!— __layout.svelte —>
<script>
  import '$lib/styles/style.scss'
</script>
```

Ok, we've gotten a pretty good idea of project structure and styling at this point. Let's move on to the last big missing piece: support for Markdown.

MOVE

Adding Markdown to SvelteKit with mdsvex

If you've heard of MDX (generally used in React-based projects), you could think of the curiously named mdsvex as the Svelte equivalent.

> The word "svelte" does not make for smooth portmanteaus, but that doesn't slow the Svelte community down much.

Anyway, mdsvex powers everything we might want to do with Markdown:

- mdsvex handles converting Markdown to HTML;
- It also allows Markdown files to be used as components; and
- Finally, mdsvex lets us use Svelte components *inside* Markdown.

First, install mdsvex:

Terminal

```
npm i -D mdsvex
```

Next, we'll need to add mdsvex to our config. Open `svelte.config.js`, and modify the code like so:

JavaScript

```
// svelte.config.js
/* Other imports here */
import { mdsvex } from 'mdsvex'

const config = {
  kit: { /* Kit options here */ },

  extensions: ['.svelte', '.md'],

  preprocess: [
    sveltePreprocess(),
```

```
    mdsvex({
      extensions: ['.md']
    })
  ]
}
```

If you'd like to get into the details of that config:

- The `extensions` config property tells Svelte what types of files to treat as components (allowing them to be imported and used in the same way Svelte components are);
- The `mdsvex()` function preprocesses Markdown to HTML—but it only targets `.svx` files by default, so we modify that as an argument.

Reminder: be sure to restart your dev server after making config changes.

If you're running into errors, double-check that everything installed properly, all the necessary imports are present in your config file, and there are no syntax errors in the config file.

Markdown pages

Because of the config changes we just made, Markdown files can now be imported and used just like Svelte components.

Thanks to mdsvex, we can now use Markdown files as pages.

Just to try it out, create `src/routes/uses.md` and throw some Markdown into it. You'll be able to see your content by visiting `/uses`:

Project files

src

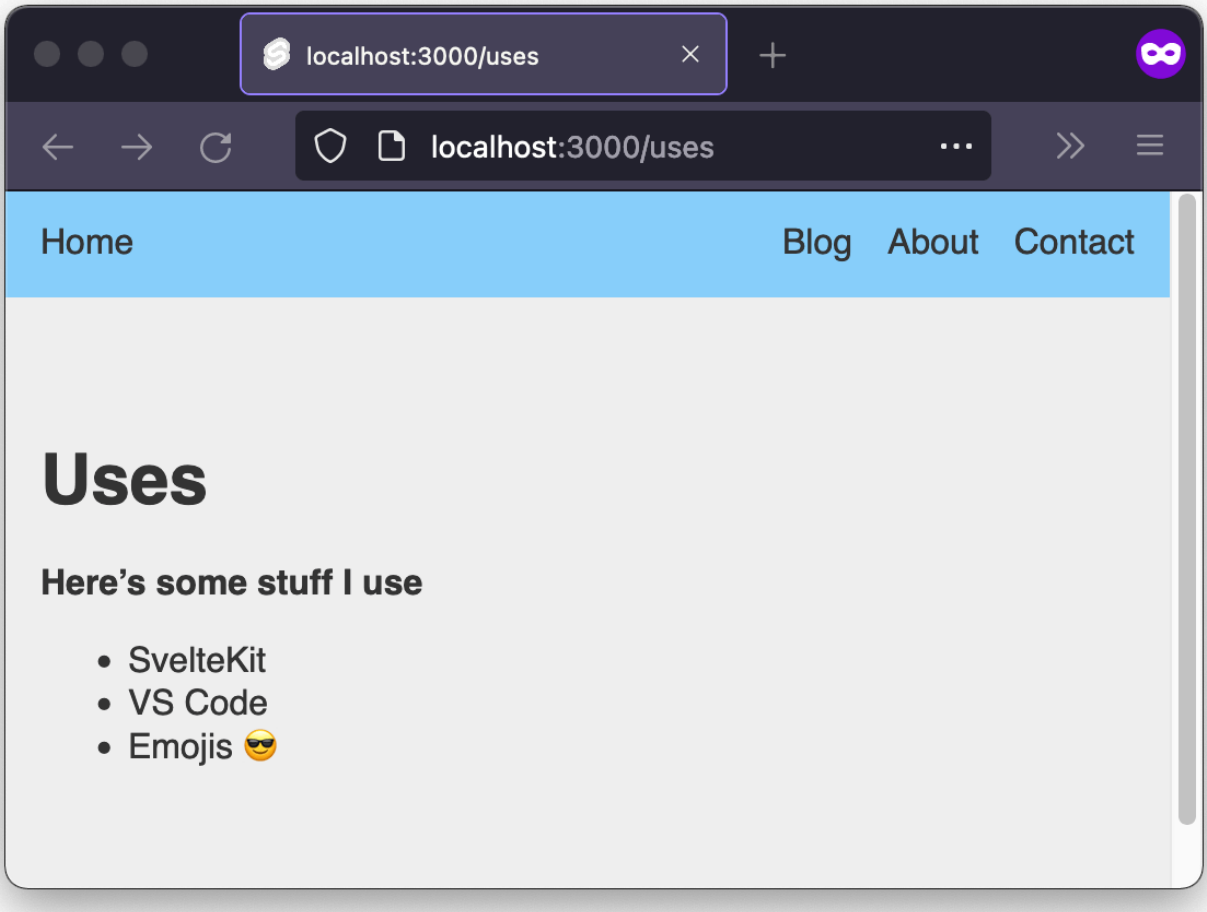
```
└ 📂 routes
  └ 📜 uses.md
```

Markdown

```
# Uses

**Here's some stuff I use**

- SvelteKit
- VS Code
- Emojis 😎
```



You can even import and use Markdown content *inside of* other files, if you want to — for example, if part of the page needed state or interactivity, but part could

to—for example, if part of the page needed state or interactivity, but part could just be Markdown text.

MOVE

Markdown pages are a handy option for text-based content, since Markdown is generally easier to author than HTML.

This `/uses` page doesn't have to be permanent, but keep it around for now; we'll reference it again in a little bit. But first, let's cover a couple other nice features of mdsvex.

Code blocks in markdown

Handily, mdsvex comes with prism.js pre-installed for code syntax highlighting. All you need to do is note the language (by adding it to the opening triple backticks), and mdsvex will process it appropriately.

Markdown

```
```js
// JavaScript goes here,
// And will be syntax-highlighted!
```
```

All the code blocks on this website are made using this technique. And as a bonus: mdsvex even has `svelte` available as a language option!

Just note that you'll need some CSS to handle the coloring, but many themes are available to either use as-is, or as a base for your own styles.

Using Svelte components in Markdown

As mentioned before: mdsvex allows you to use Svelte components *inside* of Markdown!

To do so, just drop in a `script` tag at the start of the content (MOVE r t frontmatter), and import the component within. You can then add the component to your Markdown just like you would anywhere else:

```
Markdown

---
# frontmatter goes here
---
<script>
  import SomeComponent from '$lib/components/SomeComponent.svelte'
</script>

# Markdown content here

<SomeComponent />

More markdown _here_!
```

SIDE NOTE:

As you might have already realized: this means you can even import Markdown files into other *Markdown files!*

Setting up the blog

Now that we have Markdown processing all set up, it's time to get into the real meat of this tutorial, and begin setting up the blog.

Thanks to SvelteKit's flexibility, there are actually a few ways to do this, each with their own tradeoffs. But ultimately, they all work well (especially when pre-rendering), so I've chosen the simplest.

Adding some Markdown posts

Going forward, it will be handy to have at least a couple of posts to work with, so let's create some inside of `src/routes/blog`. (If you already have some Markdown post files, feel free to just copy those over instead.)

No need for anything fancy; just at least a couple of `.md` files, with a tiny bit of Markdown in them for us to see. Something like this will work just fine:

Project files

```
src
└ routes
  └ blog
    ├ 1.md
    └ 2.md
```

Markdown

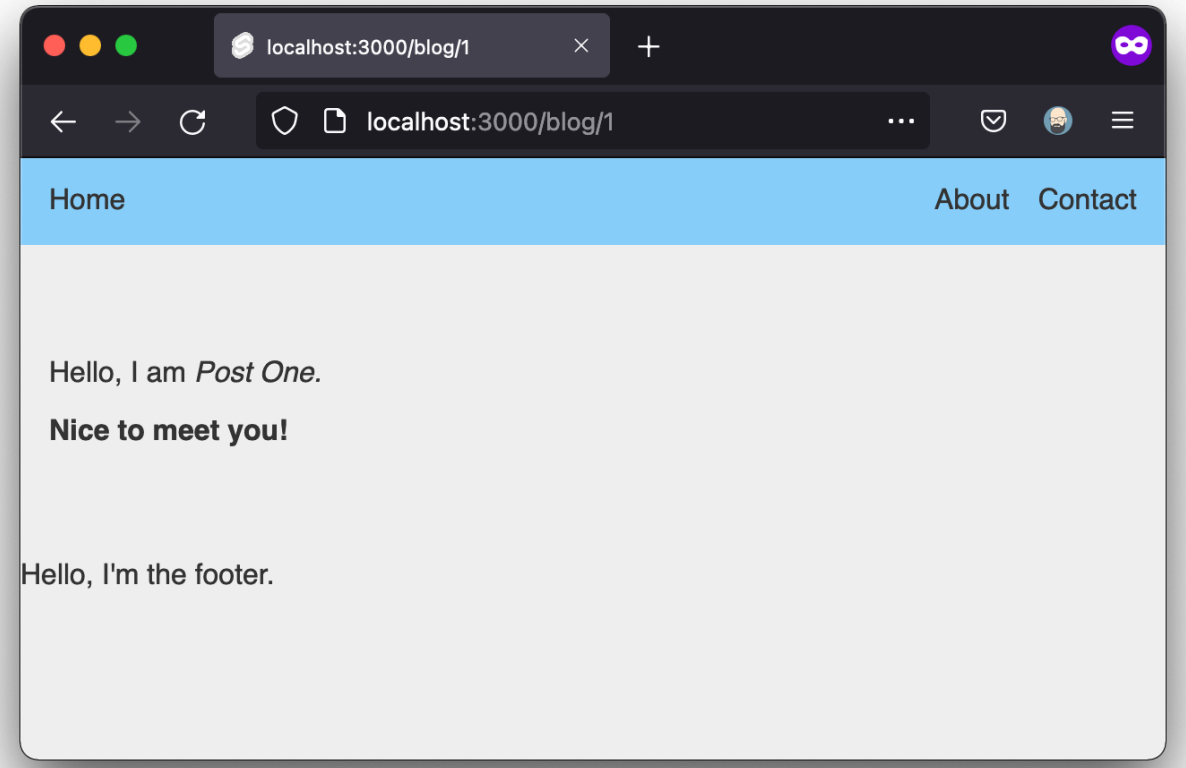
```
---
title: Post One
date: "2021-12-14"
---

Hello, I am _Post One._

**Nice to meet you!**
```

The names and content aren't important, but we *will* be using the `title` and `date` frontmatter properties later, so be prepared.

You can now visit `/blog/x` (where `x` is the slug of a blog post file) to see the post content live on the site:



Well that was easy!

However, what we're seeing above is only the content. We *also* want to display the post's frontmatter properties. And for that, we'll need an mdsvex layout.

Adding a mdsvex layout

mdsvex lets us designate a layout—that is, a Svelte component—to render Markdown files, much the same way that SvelteKit layouts render pages.

The two are similar, but not to be confused. Unlike SvelteKit layouts, there's no convention to follow with the naming or placement of mdsvex layouts. It's all explicit config.

So since it's our choice, we'll name our layout `_post.svelte`, and put it in `src/routes/blog` (just to keep related things close together).

Project files

```
src
└ routes
  └ blog
    └ _post.svelte
```

Why the underscore? Any route that begins with an underscore is private, i.e., excluded from the site's routes. Since this layout file will just serve as a template and won't have content of its own, we don't want it to have its own route or be directly visited.

Once the file is created, head back to `svelte.config.js` and add a `layout` property to the `mdsvex` function options.

There are a few ways to designate mdsvex layouts, but I find named layouts handiest.

If a named layout shares the name of the current directory, mdsvex will use it automatically.

Here's how to create one (named "blog" in this case, since we want this layout to apply inside the `blog` folder):

JavaScript

```javascript
// svelte.config.js

/* Imports here */

const config = {
  /* ...Other config properties here */
```

```
  preprocess: [
    sveltePreprocess(),
    mdsvex({
      extensions: ['.md'],
      layout: {
        blog: 'src/routes/blog/_post.svelte'
      }

    })
  ]
}
```

(*Note that the file path must be relative.*) And as always, be sure to restart the dev server after making the change.

Now, inside the file we designated as our mdsvex layout, all frontmatter properties are available as component props. We just need to name and call them, like so:

Svelte

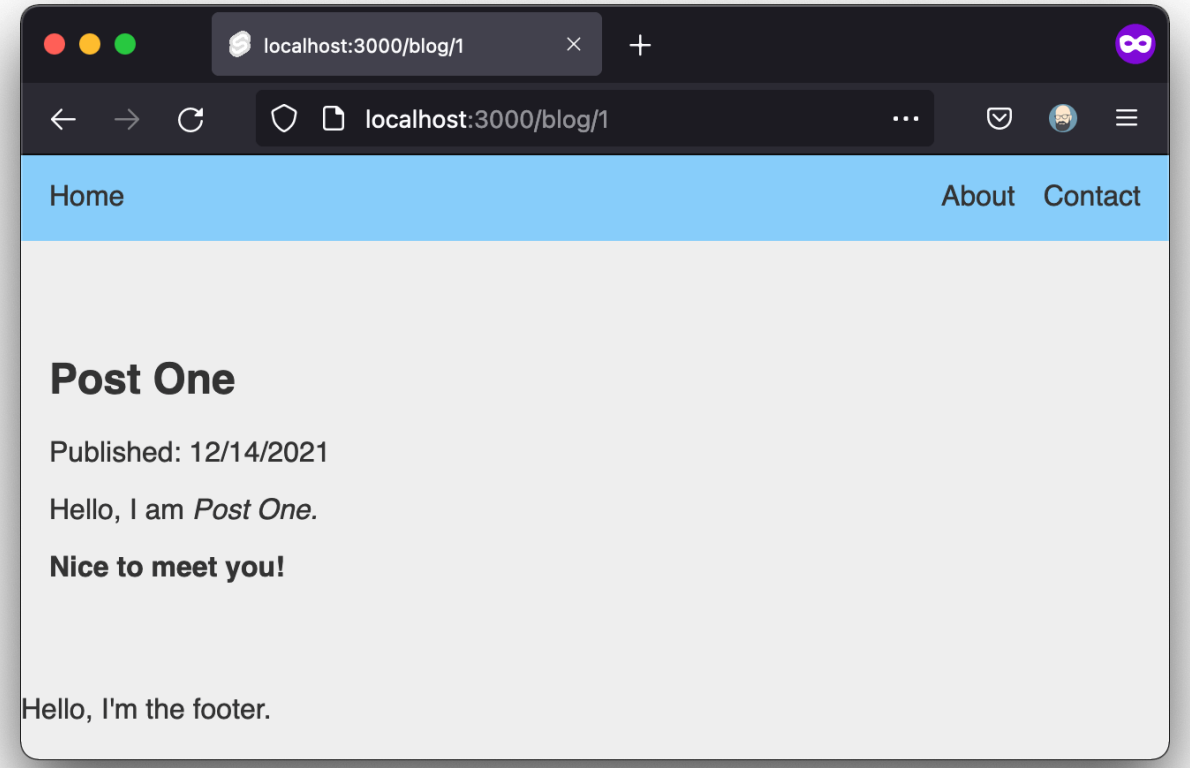
```
<!— _post.svelte —>
<script>
  export let title
  export let date
</script>

<h1>{title}</h1>

<p>Published: {date}</p>

<slot />
```

SIDE NOTE:

It might be a good idea to set defaults for each prop, in case it's missing from the frontmatter. Also, be sure the layout has a `<slot />`*, or the file's contents won't be shown. And finally, dates are tricky. If your dates aren't strings in the Markdown, you might need to add some formatting and manipulation.*

That in place, now when we load a blog post, we should see the layout pulling the

frontmatter properties in:



However, thanks to our named layout matching *only* `blog` routes, if we load the `/uses` page, we'll see the same layout does *not* apply there. Pretty awesome! (*Yes, that's all I wanted us to save it for. You can delete it now if you want.*)

Worth knowing: you can opt a Markdown file out of any mdsvex layout by adding `layout: false` to the frontmatter. You can *also* set up a fallback, in case no route matches; see the mdsvex layout docs for details.

TRADEOFFS AND OTHER OPTIONS

Using mdsvex layouts and dropping Markdown files directly into `src/routes/blog` is the simplest way to render individual posts. The main *drawback* is: unfortunately, as of this writing, mdsvex layouts can't use Sass or TypeScript (as nearly as I can tell, at least). The compiler seems to break when I try.

That might just be my config setup, but even aside from that, you're still a little more bought in to mdsvex with this approach. (Not that you wouldn't be otherwise, but it's one more feature you're relying on.) Neither of those tradeoffs may be deal-breakers for you, but they're still tradeoffs.

The other possible good approach would be to use a dynamic `[post].svelte` route, and tuck all your posts away in a hidden folder. That's a little more complex, but also perhaps a bit more flexible. We'll cover dynamic routes more later, and the same approach can easily be adapted for all blog posts, if you want. For now, I just wanted to mention the tradeoffs.

You could also use a nested SvelteKit layout, but I don't like that option because it will try to work on both our individual post pages *and* our blog index page, and we don't want that.

All right, at this point, we've got a good handle on individual posts. Now, it's time to build out the blog index page, where we'll list *all* our posts. To do that—because we'll likely want to list posts in multiple ways in various places—we'll build out an API endpoint for posts.

SvelteKit endpoints

Up until now, every route we've created has been a page. But SvelteKit offers *another* type of route, too: endpoints.

Endpoints work the same way as pages, as far as routing. But

where pages return HTML, an endpoint returns data.

There are just three important conventions to follow when creating an endpoint with SvelteKit:

1. **An endpoint's route should include the data type it returns.** For example, `/api/posts.json`. (Endpoint data is usually JSON, but it doesn't have to be.)
2. **An endpoint should export a function for each HTTP verb it accepts.** This will often just be a `get` function, but you can also use `post`, etc.
3. **Endpoint functions should return an object with `status` and `body`.** (Returning nothing is a 404 by default.)

Why an endpoint? Because we might have other places in our app we want to access these posts, too.

We *could* just put the necessary code straight in where we need it instead of going to the trouble of making an API endpoint. But unless your blog is extremely simple, odds are, you'll want to grab posts in more than one place. Having an endpoint reduces the overhead of rewriting that code to go grab our posts all over the place.

Anyway, there's more detail in the endpoint docs, but it's less complicated than it sounds, so let's dive in.

Creating an endpoint for our posts

Since we might decide to add more endpoints later, let's create a `src/routes/api` folder, and inside it, a new file named `posts.json.js`.

Project files

```
src
└ routes
```

```
api
posts.json.js
```

SIDE NOTE:

The file extension is removed from the end of all routes, so this endpoint will just be `/api/posts.json`.

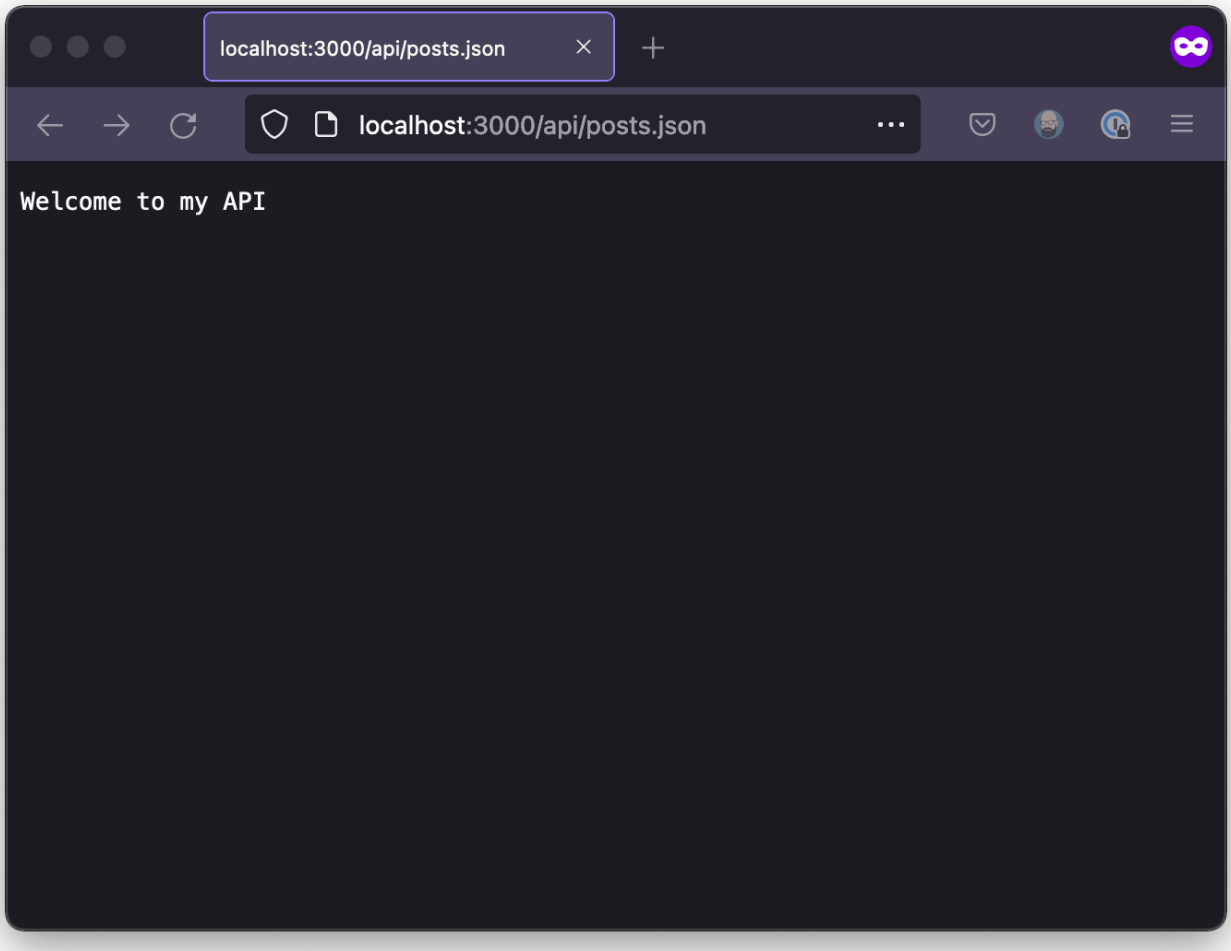
Inside this new file, we'll need to export a `get` method (since "get" is the HTTP verb we'll accept; this endpoint will only return data, not accept it).

Let's do a quick test, just to see it in action:

JavaScript

```javascript
// posts.json.js
export const get = () => {
  return {
    status: 200,
    body: "Welcome to my API"
  }
}
```

With that in place, we should be able to visit `/api/posts.json` and see the following (unimpressive, but neat!) JSON loaded in the browser:



How *cool* is it that!? Our API route doesn't return anything useful just yet, obviously, but it's still pretty awesome to have it up and running that easily!

SIDE NOTE:

While we won't add it here, you can add an argument to `get`*, which will pass in all the headers, query parameters, and lots of other info about the request.*

MOVE

Returning posts from an endpoint

The next step is to actually get data from Markdown files, and return it in the API endpoint. We'll be leaning directly on Vite for that (the build tool that powers SvelteKit under the hood).

This is going to be a lot, but we'll break it down in a second.

Inside our `posts.json.js` file, we'll put the following code:

JavaScript

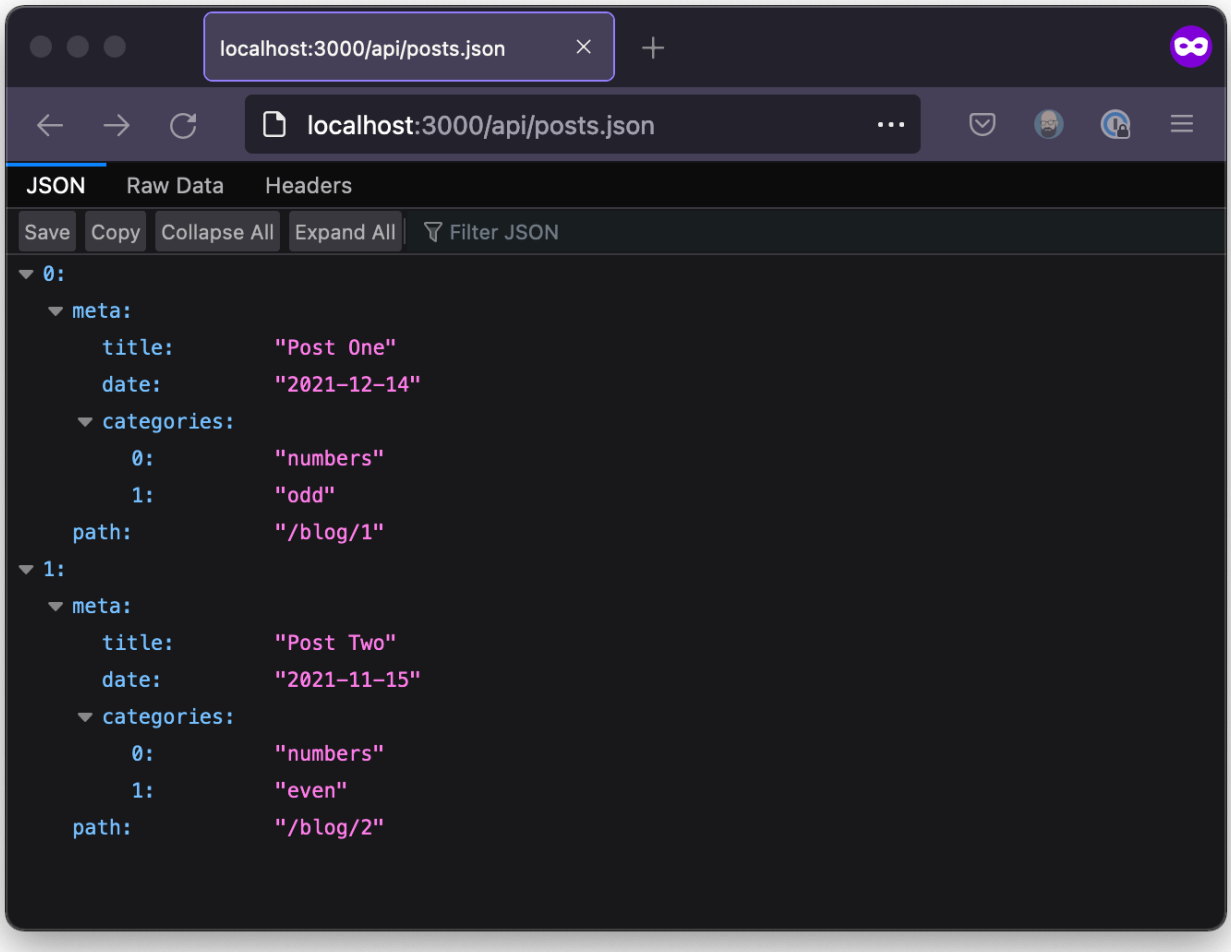
```
// posts.json.js
export const get = async () => {
  const allPostFiles = import.meta.glob('../blog/*.md')
  const iterablePostFiles = Object.entries(allPostFiles)

  const allPosts = await Promise.all(
    iterablePostFiles.map(async ([path, resolver]) => {
      const { metadata } = await resolver()
      const postPath = path.slice(2, -3)

      return {
        meta: metadata,
        path: postPath,
      }
    })
  )

  const sortedPosts = allPosts.sort((a, b) => {
    return new Date(b.meta.date) - new Date(a.meta.date)
  })

  return {
    body: sortedPosts
  }
}
```

Whoa! Again, that might look like a lot, but when you consider it's actually *everything* we need to create an API endpoint to return *all* our site's posts, it's kind of remarkable how few lines that is.

Let's go over what's happening in that code:

- `import.meta.glob` is a Vite function. It imports any files that match the glob (wildcard string) provided—in this case, all `.md` files inside `src/routes/blog`.
  - That function returns an object where each file's relative path is the key, and the value is a "resolver" function (my term; not official) that loads the file contents as a JavaScript promise.
- The `map` method is there to shape each file's data, so it's easier to work with on the front end. And since each item waits for a promise, we wrap it in an `await Promise.all`.
  - Since we know the path will begin with `..` and end with `.md`, we can safely use `.slice(2, -3)` to remove those characters and end up with the route.
- Next, we sort the posts by descending date (since this is a blog, of course, and we'll want our newest posts showing first).
  - Note that your posts will need a validly formatted `date` frontmatter property for the sorting to work—i.e., something `new Date()` can understand.
- Finally, we `return` the finished product as the `body` of our API response. (The 200 status code is implicit here, since we're successfully returning a `body`.) SvelteKit handles the conversion to JSON.

Let's try it out! Refresh your `/api/posts.json` path now, and you should see some *real* data!



SvelteKit handles converting the response to JSON and setting the proper headers automatically.

Even cooler: this endpoint will update automatically with each Markdown post we add!

Finishing the blog index page

Now that we have an endpoint for all our posts, building out our blog index page will be a piece of cake!

We *could* do the loading client-side, with a `fetch` call in an `onMount` function. That would work. But it would *also* result in a loading state with no visible content, which isn't a great user experience. So instead, let's use SvelteKit's built-in server-side rendering.

Server-side rendering with `load`

Page and layout components can have a `load` function, which runs *before* the component is created—making it perfect for fetching API data, such as our blog posts.

Because it runs on the server, the `load` function is called from its own distinct `<script>` tag, differentiated with a `context="module"` attribute:

Svelte

```
<script context="module">
  export const load = () => {
    // Runs before the component is created
  }
</script>

<script>
  // This is the "normal" client-side script
</script>
```

Other important things to know about the `load` function:

- **It has access to a few special arguments** (including `url` and `params`, which contain contextual info about the request).

- **It should return an object**. Among other things, this can include a status code, and props for the component to use.
- **It runs both server- and client-side.** That's mostly unimportant when prerendering to static files, as we are. Still, `load` shouldn't reference environment-specific things, like `window` or `process`.

I already mentioned `url` and `params`, but one other argument the `load` function has access to is `fetch`, which allows the browser's `fetch` function to be used on the server. (Ordinarily, `fetch` is browser-only.)

Svelte

```
<!— src/routes/blog/index.svelte —>
<script context="module">
export const load = async ({ fetch }) => {
  const posts = await fetch('/api/posts.json')
  const allPosts = await posts.json()

  return {
    props: {
      posts: allPosts
    }
  }
}
</script>
```

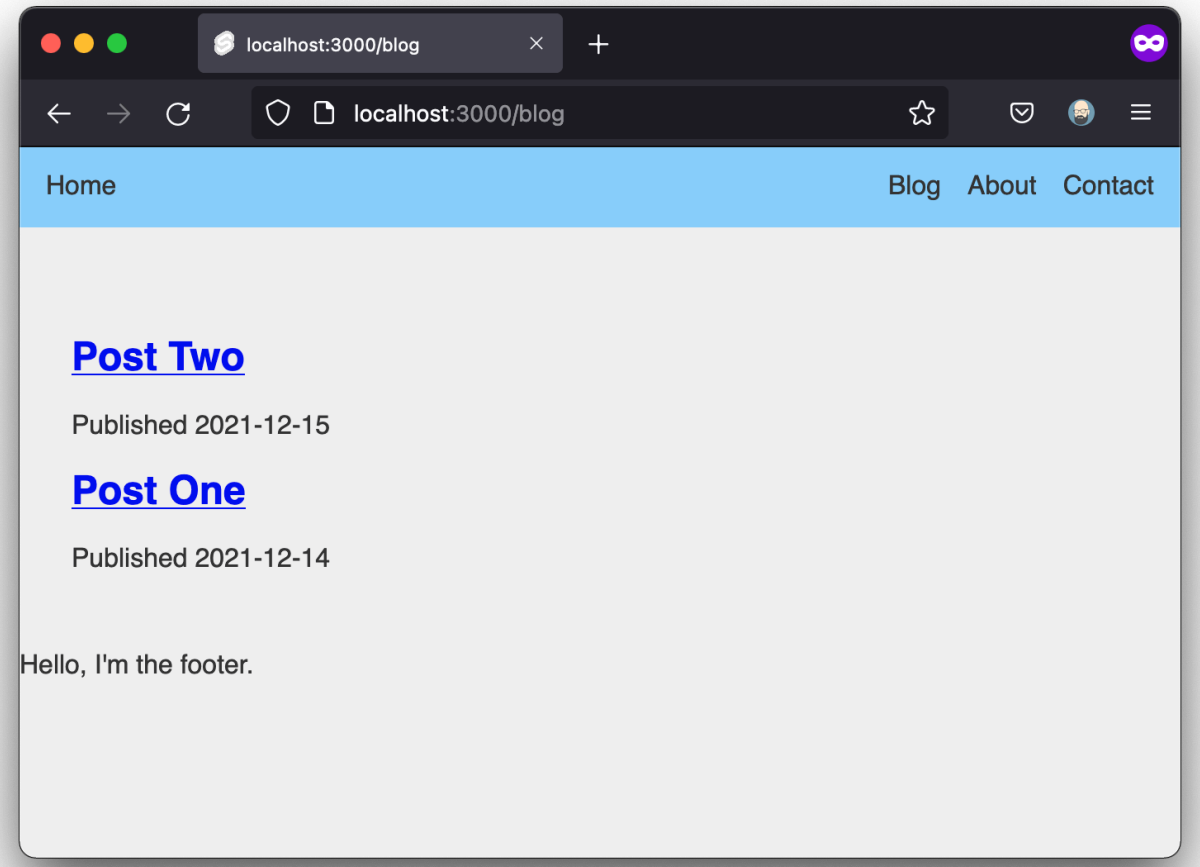
That bit of server-side loading handles everything we need! Now we've got a `posts` prop being passed to the component, and we can use it to loop over and render posts on our blog index page.

Svelte

```
<!— src/routes/blog/index.svelte —>
<script context="module">
  // Our `load` function here
</script>

<script>
  export let posts
</script>

<ul>
```

```
    {#each posts as post}
      <li>
        <h2>
          <a href={post.path}>
            {post.meta.title}
          </a>
        </h2>
        Published {post.meta.date}

      </li>
    {/each}
  </ul>
```

The HTML in the above example is a bit simple (and doesn't properly account for a scenario in which there are no posts to display), but hopefully you get the gist. You may want to add other code, based on the frontmatter in your own blog posts. (One thing definitely missing is a featured image/image URL, but you could also have excerpts or other content as well.)



MOVE

Prerendering with the static adapter

SvelteKit is server-rendered by default. That's great, because server-side rendering is generally better for performance, accessibility, and SEO. However, that also means static file prerendering (which is what we want) is opt-in.

You *can* manually make any page prerendered—handy for things like an "about" or "faq" page that won't have any dynamic content. However, if we want our *entire* site to be statically pre-rendered, marking each and every page by hand would be an error-prone chore. So instead, we'll reach for SvelteKit's static adapter.

SvelteKit has a number of available adapters, which shape our project's build output based on how we'd like to deploy.

SvelteKit has many adapters, and comes pre-installed with one that will automatically detect and build properly for Netlify, Vercel, or Cloudflare Pages—which is both handy and impressive! But for static pre-rendering, we want `adapter-static`.

Terminal

```
npm i -D @sveltejs/adapter-static@next
```

Once installed, we have to make a couple small changes to our `svelte.config.js` file to use the static adapter.

First, change `adapter-auto` to `adapter-static` (in the import near the top of the file):

JavaScript

JavaScript

```
// Replace the original `adapter-auto` line with this in svelte.config.js
import adapter from '@sveltejs/adapter-static'
```

And second, set `config.kit.prerender.default` to `true`, to make prerendering the default. `config.kit` should already exist, but you'll need to add the rest. It should look like this when you're done:

JavaScript

```
// svelte.config.js
const config = {
  // ... Other properties here

  kit: {
    // ... Other properties here, too
    prerender: {
      default: true
    },
  }
}
```

SIDE NOTE:

The `prerender` prop requirement seems to be relatively new to SvelteKit, and I was able to get some builds to work without it, but others, not. I included it just to be safe.

You *can* supply further customization options if you want to (as an object passed to the `adapter()` function), but the defaults are great for us, so there's no need. You can check out the adapter-static readme for more if you like.

With our adapter installed and in use in our config file, every page on our site will be pre-rendered as static HTML at build time! You can even view source in

the browser to confirm that the site is now served as pre-rendered HTML.

SIDE NOTE:

One of the great things about SvelteKit's static mode is that it pre-renders all your pages as static HTML, but also *loads some client-side JavaScript to help with navigation and preloading in the background. This way, the site works just fine without JavaScript, but is progressively enhanced.*

One final note: **you wouldn't *have* to use the static adapter at this point.** You could just skip that part and go with the default adapter. That would allow you to deploy the site to Netlify, Vercel, or Cloudflare Pages, and the site would be server-rendered using serverless functions, instead of statically pre-generated.

Both would work great, and which to go with depends mainly on your site and its needs. I just bring it up because it's awesome that SvelteKit can completely transform from one thing to another simply by swapping out the adapter.

Optional finishing touches and extra features

At this point, our blog is up and running. You could take it and run with it in its current shape, and begin adding your own styling and customizations.

The rest of this post will focus on possible enhancements and diving a little deeper into some aspects of SvelteKit.

However, what we have is basic and there some extra enhancements we may want to make.

Add dynamic post category pages

Many blogs use taxonomies like categories and tags to let the reader sort and view posts. SvelteKit lets us accomplish this with dynamic pages.

> You could think of a dynamic page as a wildcard route. Essentially, it's a template that processes a route as an argument.

Dynamic pages are useful because we might want one template file to handle any number of potential routes. In our case, we'll have one file to handle *any* `/blog/category/*` route, no matter what the `*` might be.

To start, if you don't already have some categories in your posts' frontmatter, it's a good idea to add some now. I did this, just for the sake of example:

Markdown

```
<!-- 1.md -->
---
categories:
  - "numbers"
  - "odd"
---
```

Markdown

```
<!-- 2.md -->
---
categories:
  - "numbers"
  - "even"
---
```

To make a page dynamic, all we have to do is wrap its title in brackets. So, for categories, we'll add a `categories/[category].svelte` route in `src/routes/blog`.

MOVE

Project files

```
src
└ routes
  └ blog
    └ categories
      └ [category].svelte
```

SIDE NOTE:

The word `category` *isn't special; it's just a variable. But as with any variable, it's good to name it semantically.*

Once you've created that file, you may notice you can actually visit `/blog/category/` followed by anything, and you won't get a 404. The dynamic page handles *all* unmatched `/blog/category/` routes. The trick now is just to load the right content based on the route.

Just to get an idea of what we're working with, let's start with any JavaScript developer's best friend: `console.log`.

Svelte

```
<!— [category].svelte —>
<script context="module">
  export const load = ({ params }) => {
    console.log(params)
    return {
      status: 200
    }
  }
</script>
```

Notice if you load a blog category page now, you can see `params` in the browser console, as well as in the terminal where your local dev server is running:

Terminal

```
{ category: 'numbers' }
```

This confirms our `load` function is running both server- and client-side. The current route is called `params.category`, because `category` is what we named our dynamic page, and whatever category route we're visiting will be its value.

SIDE NOTE:

Also note that you won't see what you logged in the browser console if you hit an error; that's why we added the explicit 200 status, just for demo purposes.

Knowing that the current `/blog/categories/*` route will be available as `params.category`, we can get to work. We'll use that, and our existing API endpoint, to filter posts:

Svelte

```
<!— [category].svelte —>
<script context="module">
  export const load = async ({ params, fetch }) => {
    const currentCategory = params.category
    const response = await fetch('/api/posts.json')
    const posts = await response.json()

    const matchingPosts = posts
      .filter(post => post.meta.categories.includes(currentCategory))

    return {
      props: {
        posts: matchingPosts
      }
    }
  }
</script>
```

I won't go into how to render the matching content, but it's pr[illegible] m[illegible] the same as our blog index page. (In fact, you could easily create a [illegible]nt to handle listing posts, and reuse it in both places!)

It's probably a good idea to wrap that code in a `try` / `catch` block—and for that matter, to anticipate situations where no posts will match the given category, and handle that properly in the UI. (An `{#if posts.length}` block with an `{:else}` should do the trick.)

Inside the `_post` template, listing a post's categories just requires grabbing the prop and looping over it:

Svelte

```
<!— _post.svelte —>
<script>
  // ... Other props here
  export let categories;
</script>

<!— ...Post HTML here —>

{#if categories.length}
  <aside>
    <h2>Posted in:</h2>
    <ul>
      {#each categories as category}
        <li>
          <a href="/blog/categories/{category}">
            {category}
          </a>
        </li>
      {/each}
    </ul>
  </aside>
{/if}
```

Finally, if you *really* wanted to go the extra mile, you could add an `index.svelte` file to `src/routes/blog/categories/`, to display *all* categories. I won't go into that here, since it's an additional feature that won't benefit all blogs, however.

MOVE

Implement page transitions

If you want to get fancy, adding a page transition in SvelteKit is pretty simple! Let's look at the code, then we'll go over each part of it.

Svelte

```
<!— __layout.svelte —>
<script context="module">
  export const load = ({ url }) => {
    const currentRoute = url.pathname

    return {
      props: {
        currentRoute
      }
    }
  }
</script>

<script>
  // ...Other imports here
  import { fade } from 'svelte/transition'

  export let currentRoute
</script>

<!— Other HTML here —>

{#key currentRoute}
  <main in:fade={{ duration: 150, delay: 150 }} out:fade={{ duration: 150 }}>
    <slot />
  </main>
{/key}
```

- To start, in our layout, we'll need `load` to grab the current route. We'll pass that as a prop named `currentRoute`, to be used by the component.
- We'll use `fade` from the Svelte transition library as our transition, though you could choose another if you like. The main thing is just to add `delay` to the

`in` transition, so it doesn't start before the old page is done transitioning out.

- Finally, we'll wrap the page contents in a `#key` block.

A `#key` block takes an expression, and automatically re-renders its contents whenever that expression changes.

In this case, that will be the current page route, so that we can re-render the page itself every time the route changes.

That's all it takes! Transitions are an incredibly powerful part of Svelte, and the `svelte/transition` library offers several options besides `fade`. Feel free to play around with them.

Add an RSS feed

Remember, endpoints don't have to return JSON; they can return XML, too. This handy fact makes it dead simple to implement an RSS feed on our blog!

To start, create an `rss.xml.js` file in your `routes` folder, to make the RSS feed available at the route `/rss.xml`.

Project files

```
src
└ routes
  └ rss.xml.js
```

Inside that file, we'll use `import.meta.glob` again, and convert the result to XML.

SIDE NOTE:

Sadly, we can't use `fetch` *in an endpoint, but now that we're using* `import.meta.glob` *in two places, we could abstract it to a helper function to cut down on repeated code.*

JavaScript

```
const siteURL = 'https://your-domain.tld'
const siteTitle = 'Your site title here'
const siteDescription = 'Your site description here'

export const get = async () => {
  const posts = await Promise.all(
    Object.entries(import.meta.glob('./blog/*.md')).map(async ([path, resolve
      const { metadata } = await resolver()
      const slug = path.slice(2, -3)
      return { ...metadata, slug }
    })
  )
  .then(posts => {
    return posts.sort((a, b) => new Date(b.date) - new Date(a.date))
  })

  const body = render(posts)
  const headers = {
    'Cache-Control': 'max-age=0, s-maxage=3600',
    'Content-Type': 'application/xml',
  };

  return {
    body,
    headers,
  }
}

const render = (posts) =>
`<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0" xmlns:atom="http://www.w3.org/2005/Atom">
<channel>
<title>${siteTitle}</title>
<description>${siteDescription}</description>
<link>${siteURL}</link>
<atom:link href="${siteURL}/rss.xml" rel="self" type="application/rss+xml"/>
${posts
  .map(
    (post) => `<item>
<guid isPermaLink="true">${siteURL}/blog/${post.slug}</guid>
<title>${post.title}</title>
```

```
<link>${siteURL}/blog/${post.slug}</link>
<description>${post.excerpt}</description>
<pubDate>${new Date(post.date).toUTCString()}</pubDate>
</item>`
  )
  .join('')}
</channel>


</rss>
`
```

Please note that the above code block will need some modification! The example above is more or less straight from this site, and may not be the right shape for your needs. At the very least, you'll need to replace the URL and text placeholders, but you may also need to update the routes and the post frontmatter properties being referenced.

I pulled my example from this guide and this one, for reference. And if you want to make sure you've done it correctly, here's an online XML validator.

Add heading links with rehype

It's nice to let users link directly to a section of a post. Manually adding links to all our headings would be tedious, however, and goes against the point of writing in Markdown.

Luckily, mdsvex allows us to use many rehype plugins to add extra features to markdown processing, and so we can make this all happen automatically with just a bit of config adjustment.

SIDE NOTE:

Rehype is an HTML parser, and not specific to Svelte or mdsvex; it's commonly used in a wide range of projects.

There are two rehype plugins we'll want specifically:

- `rehype-slug`, which automatically adds IDs to headings;
- `rehype-autolink-headings`, which (as you can guess from the name), automatically adds links to our headings.

Install them both like so:

Terminal

```
npm i rehype-slug rehype-autolink-headings
```

Then we'll pop open our `svelte.config.js` file, import our two new rehype plugins, and add them to the `mdsvex` config options object, like so:

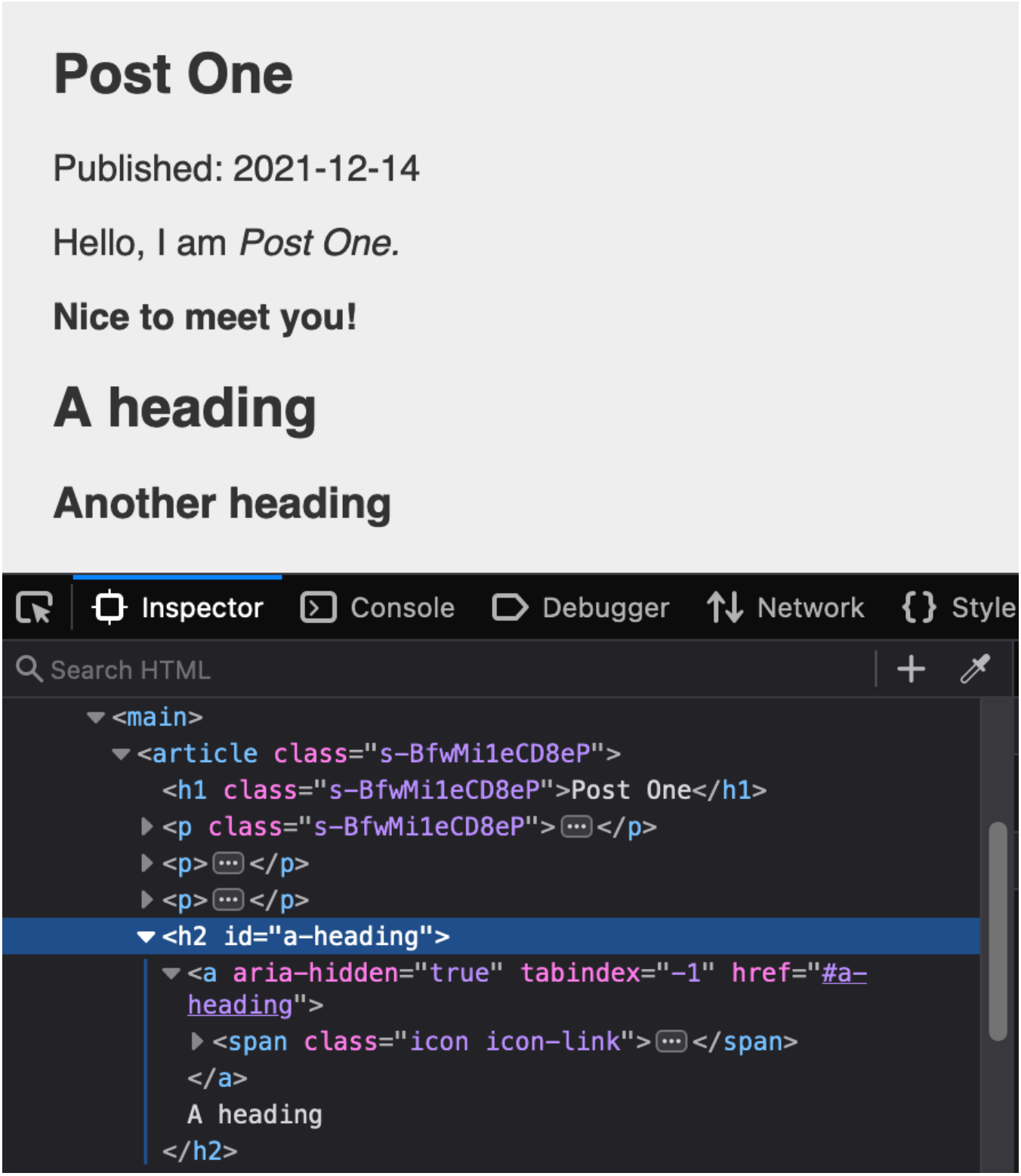
JavaScript

```javascript
// svelte.config.js
/* Other imports here */
import rehypeSlug from 'rehype-slug'
import rehypeAutolinkHeadings from 'rehype-autolink-headings'

const config = {
  /* Other config properties here */

  preprocess: [
    sveltePreprocess(),
    mdsvex({
      extensions: ['.md'],
      rehypePlugins: [
        rehypeSlug,
        rehypeAutolinkHeadings,
      ]
    }),
  ],
};

export default config;
```

Note that the plugins **must** go in that order! `rehypeSlug` adds IDs to our headings, and `rehypeAutolinkHeadings` only works on headings that have IDs.

(Fun fact: I discovered these two were in the wrong order on this s hi iting this, when I suddenly realized it was only half working. Writing technical posts is a great way to find out how much you missed.)

With that in place, restart the dev server. Now pop open the inspector and check out an `h2` through `h6` generated from Markdown, and we'll see some additions:



```
<h3 id="another-heading">
  <a aria-hidden="true" tabindex="-1" hre#another-
  heading">…</a>
```

However, we'll need to add a bit of styling to make the links show up properly.

SIDE NOTE:

The `.icon.icon-link` *class is a Font Awesome convention, so it doesn't mean or do anything in our project by default.*

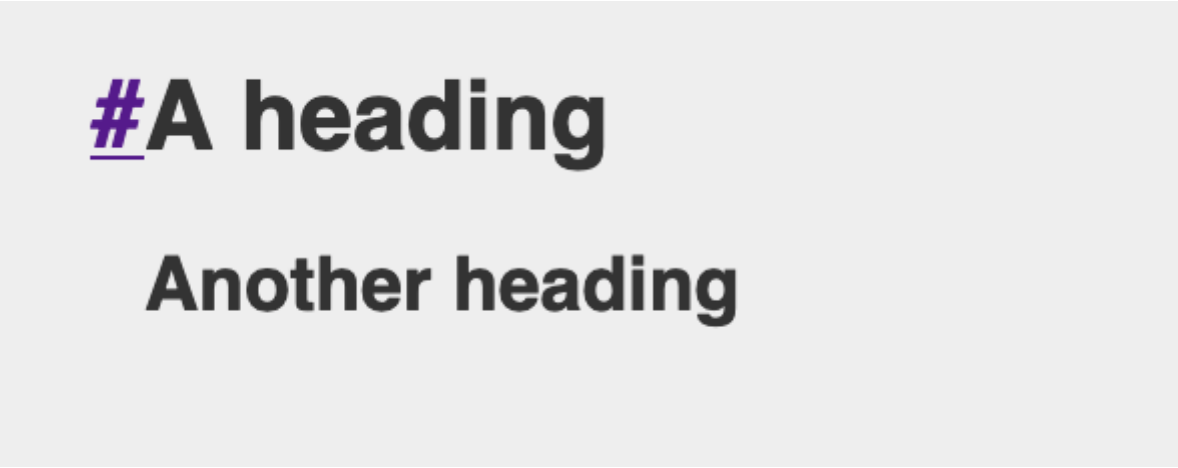
How you choose to handle this is personal preference, but this block of Sass should basically do the trick:

Code

```
:where(h2, h3, h4, h5, h6) {
  .icon-link {
    position: relative;

    &::before {
      content: '#';
      position: absolute;
      left: -1ch;
      top: 0;
      line-height: 1;
      opacity: 0;
      text-decoration: underline;
    }
  }

  &:hover .icon-link::before {
    opacity: 1;
  }
}
```

That CSS will make a pound sign (or hash, or "octothorpe," if you're fancy) appear whenever the user hovers on a heading with a `.icon-link` in it:

MOVE



Thanks to how CSS treats pseudo elements, that icon is fully clickable as part of the link, to navigate directly to the heading in question.

SIDE NOTE:

Please be aware that while this example is ok, hovering is not an intuitive gesture on touchscreen devices. It's probably better to always *show some indication of the link.*

You could *also* add some JavaScript to handle automatically copying the link to the clipboard (probably with some JavaScript inside a Svelte component's `onMount` function), but I'll leave that detail up to you. For now, our links are at least present and working, even if they might not be ideal yet.

Add autoprefixer with PostCSS

Since we're already using `svelte-preprocess`, adding autoprefixer to our

toolchain is as simple as installing it, and adding it in our config file:

Terminal

```
npm i -D autoprefixer
```

Then add it in the `sveltePreprocess()` function arguments:

JavaScript

```
// svelte.config.js

/* Other imports here */
import autoprefixer from 'autoprefixer'

const config = {
  /* Other config options here */
  preprocess: [
    sveltePreprocess({
      postcss: {
        plugins: [autoprefixer]
      },
      /* Other sveltePreprocess options here, like SCSS */
    }),
    /* Other preprocessors here, like mdsvex */
  ],
}
```

That's all we need to do! Autoprefixer is now working automatically to prefix any CSS properties that might need it for maximum backwards compatibility with other browsers.

As an added bonus, if you want to add any *other* PostCSS plugins, you can now do so in just the same way.

Add page head meta tags

Ideally, we'll probably want to set up some stuff for the `<head>` on our various pages. For pages in general, but posts especially, it's important to have the right

`title` attributes, Open Graph info, share images, etc.

MOVE

Svelte has a `<svelte:head>` component for adding HTML to the page head from anywhere.

Add one somewhere in `_post.svelte` (I like to put it just before the HTML, personally), and let's add a title inside.

Svelte

```
<!— _post.svelte —>
<svelte:head>
  <title>My blog - {title}</title>
  <meta property="og:title" content={title} />
</svelte:head>
```

You should now see the title showing up properly in the browser bar.

There are many other properties you might want to add here, *especially* if you're trying to optimize your posts for share-ability. I won't go into any more detail here, but it's worth verifying all your pages and posts look good when shared with Open Graph checker tools (many are available online).

At this point, we should probably go through our pages and add `<title>` tags, etc. to them as well. For example:

Svelte

```
<!— about.svelte —>
<svelte:head>
  <title>About - Josh Collinsworth</title>
</svelte:head>

<!— ...HTML here —>
```

Note also that SvelteKit offers several other similar elements, like `<svelte:window>` and `<svelte:body>`, where we need to tap into parts of our

site or app normally outside the boundaries of our components.

Improve the posts API endpoint

The posts API is functional, but it can be improved in several ways.

For starters, we'll definitely want to put the code we just wrote into a `try` / `catch` block that returns the proper status code when things go wrong. Some refactoring might also be in order. We'll also probably want to add some other extra features to this API route eventually.

Our API is currently missing pagination options, which could be very needed depending on the number of posts.

Another possible future enhancement would be adding the post's content to the returned JSON, which we don't do currently. The "resolver" function offers a `default.render` method for that, if you so choose.

Finally, you could build in some post filtering, but it might be better to build out a dynamic endpoint for that. The SvelteKit docs section on Rest parameters is worth a look as far as that goes.

Customize the error page

What you're looking at when you hit an error is SvelteKit's built-in error component.

However, just like we can provide our own `__layout.svelte` file, we can *also* provide an `__error.svelte` file, to template and format any errors our users might run into.

I won't go into detail here, but you can check the official Svelte docs error pages

entry for more info.

Preload routes

SvelteKit is already fast, and when we pre-render our HTML like we are via the static adapter, it's going to be blazing. But we can actually push the performance *even further* with SvelteKit's built-in preloading.

SvelteKit ships with two functions for preloading data in the background: `prefetch`, and `prefetchRoutes`.

To use them, just import them in a page or layout (our global layout file might be a good choice):

Svelte

```
<script>
  import { prefetch, prefetchRoutes } from '$app/navigation'
</script>
```

SIDE NOTE:

`$app` *contains several modules available in SvelteKit for things you might need in an app, like* `env` *and* `paths`*, along with* `navigation`*. See the modules docs for full details.*

The difference between the two is:

- `prefetch` preloads a single route;
- `prefetchRoutes` preloads *all* routes.

Svelte

```
<script>
  import { prefetch, prefetchRoutes } from '$app/navigation'

  prefetch('/blog') // Loads the blog page in the background

  prefetchRoutes() // Loads ALL routes in the background
</script>
```

Because `prefetchRoutes` grabs *all* the site's data behind the scenes, we should use it cautiously and judiciously. We don't want to force users to download megabytes of data presumptively. That could very well cost them real money, for pages they might not even look at.

Add anchor options

Speaking of preloading: SvelteKit offers a slightly less greedy version of it, as one of its anchor options.

Anchor options are special, SvelteKit-specific attributes you can add to `<a>` anchor tags. There are three:

- `<a sveltekit:prefetch>` causes the link to begin preloading as soon as the user *hovers*, rather than waiting for a click, saving some milliseconds.
- `<a rel="external">` signals to SvelteKit that the link in question is *not* part of our SvelteKit app, and the router shouldn't try to handle it.
- `<a sveltekit:noscroll>` prevents SvelteKit from resetting the scroll position to the top of the new page. This is usually undesirable on websites, but may be more intuitive in some app situations.

Conclusion

This has been a long post, and we've covered a lot. I hope this has been a valuable

resource. Even if you *still* aren't sold on SvelteKit (or at least, not as a static site generator), hopefully this post has given you an idea of the broad capabilities of SvelteKit, and maybe even given you some ideas of how you might use it yourself.

I'm Josh Collinsworth, a frontend developer, designer, teacher, and writer. I currently live in Omaha, Nebraska, and work for Shopify as a senior frontend developer.

- Buy me a coffee
- Send me a note about this post
- More about me

POSTED IN:

SVELTE JAVASCRIPT WEB SASS

- Writing & Speaking
- Contact

MOVE

LINKS

- RSS
- Ko-Fi
- GitHub
- CodePen
- Twitter

©2015–2022 Josh Collinsworth.

Made with WordPress Gridsome SvelteKit; hosted on Netlify.